

Điều khiển cánh tay UR3e viết chữ trên mặt phẳng 2D

ROS 2 Humble · MoveIt 2 · Ignition Gazebo Fortress

Mục	Nội dung
Sinh viên	Việt Anh
MSSV	
Lớp	
Robot	Universal Robots UR3e (mô phỏng)
Bài toán	Vẽ một đường tròn, chữ V và chữ A trên cùng một mặt phẳng, ba hình cách nhau không vẽ chồng lên nhau
Repo GitHub	https://github.com/vanhhhh512/ur3e_va
Video demo	(điền link Google Drive, chế độ Anyone with the link)

MỤC LỤC

1. Giới thiệu đề tài	3
1.1. Yêu cầu của đề và mức đáp ứng	3
1.2. Môi trường sử dụng	3
2. Cấu trúc chương trình	3
2.1. Cấu trúc thư mục	3
2.2. Sơ đồ khối các thành phần	4
2.3. Luồng hoạt động khi chạy	4
3. Các node và package chính	5
4. Cách tạo quỹ đạo	5
4.1. Hệ tọa độ mặt phẳng vẽ và phép ánh xạ	5
4.2. Đường tròn	6
4.3. Chữ V và khung chữ A	6
4.4. Thanh ngang của chữ A	6
4.5. Kiểm tra bố trí trước khi chạy	7
4.6. Trình tự vẽ một nét	7
5. MoveIt lập kế hoạch và gửi trajectory tới robot	7
6. Những vấn đề gặp phải và cách giải quyết	8
7. Kết quả chạy thử	9
8. Hướng dẫn chạy nhanh	10

1. Giới thiệu đề tài

Bài thực hành xây dựng một package ROS 2 điều khiển cánh tay Universal Robots UR3e cầm bút vẽ lên một mặt phẳng dựng đứng trước mặt robot. Robot vẽ lần lượt ba hình: một đường tròn khép kín, chữ V vẽ liền một nét, và chữ A gồm hai nét mà giữa hai nét robot nhấc bút lên rồi hạ xuống chỗ mới. V và A là hai chữ cái đầu trong tên người thực hiện; đường tròn được giữ lại theo yêu cầu về video demo của đề bài.

Điểm đặt trọng tâm của cách làm là tách phần hình học ra khỏi phần điều khiển. Hình vẽ được mô tả bằng toạ độ hai chiều ngay trên mặt phẳng giấy, hoàn toàn không dính tới hệ toạ độ của robot; một lớp riêng lo việc quy đổi sang pose sáu chiều cho MoveIt. Nhờ vậy, muốn đổi chữ, đổi kích thước hay dời cả bản vẽ sang chỗ khác thì chỉ cần sửa một file YAML, không phải chạm vào mã C++ điều khiển.

Chương trình cũng tự kiểm tra bố trí trước khi robot cử động và tự đo sai số sau khi vẽ xong, thay vì chỉ quan sát bằng mắt.

1.1. Yêu cầu của đề và mức đáp ứng

Yêu cầu	Đáp ứng
Chạy UR3/UR3e simulation với Gazebo	Ignition Gazebo Fortress qua gói <code>ur_simulation_gz</code>
Package ROS 2 có launch file điều khiển	Package <code>ur_pen_plotter</code> , chạy bằng một lệnh <code>demo_all.launch.py</code>
Viết chữ cái đầu trong tên sinh viên	Chữ V và chữ A
Chữ nằm trong mặt phẳng Cartesian tự chọn	Mặt phẳng vuông góc trục X, cách đế 0,40 m
Tự thiết kế tập waypoint	Sinh trong <code>strokes.cpp</code> , lấy mẫu mỗi 2 mm
Chia chữ nhiều nét, nhấc đầu công tác	Chữ A gồm hai nét, nhấc bút 5 cm giữa hai nét
Dùng MoveIt 2 lập kế hoạch và thực thi	<code>MoveGroupInterface</code> , <code>computeCartesianPath</code> và OMPL
Không vượt giới hạn khớp, không tự va chạm	Giới hạn khớp lấy từ <code>ur_description</code> ; bật kiểm tra va chạm khi nội suy Cartesian
Hiển thị đường đi end-effector trên RViz	Marker nét mực và marker waypoint

1.2. Môi trường sử dụng

Thành phần	Phiên bản
Hệ điều hành	Ubuntu 22.04 LTS
ROS 2	Humble Hawksbill (Desktop)
Mô phỏng vật lý	Ignition Gazebo Fortress, plugin <code>ign_ros2_control</code>
Lập kế hoạch chuyển động	MoveIt 2 (OMPL)
Ngôn ngữ	C++ cho node điều khiển, Python cho launch và script phụ trợ

2. Cấu trúc chương trình

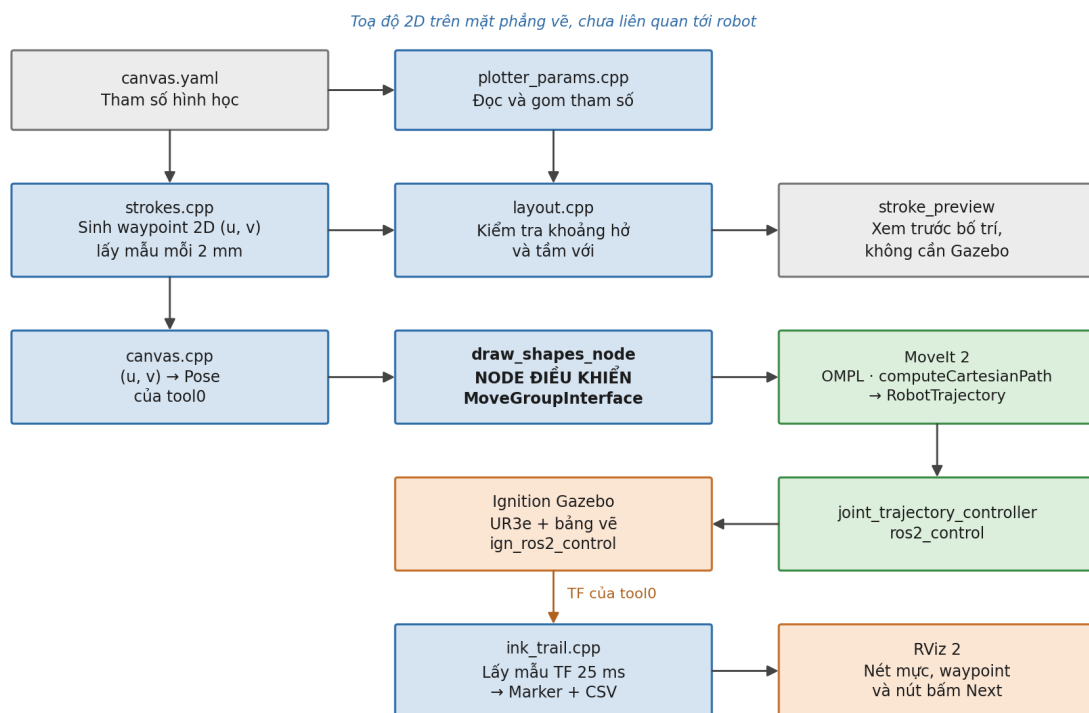
2.1. Cấu trúc thư mục

```
ur3e_va/
```

└─ README.md	hướng dẫn cài đặt và chạy
└─ bao_cao_ur3e.pdf	báo cáo này
└─ scripts/	
└─ sim_test.sh	chạy trọn bộ không cần màn hình, tự chấm sai số
└─ plot_trail.py	vẽ lại bản vẽ từ CSV, so dự định với thực tế
└─ src/ur_pen_plotter/	package do sinh viên phát triển
└─ config/canvas.yaml	toàn bộ hình học bản vẽ
└─ config/sim_controllers.yaml	controller và dung sai cho Ignition
└─ include/ur_pen_plotter/	canvas.hpp · strokes.hpp · layout.hpp ·
	ink_trail.hpp
└─ src/	draw_shapes_node.cpp · stroke_preview.cpp và
	file cài đặt
└─ launch/	demo_all · bringup_sim · draw
└─ rviz/pen_plotter.rviz	cấu hình sẵn Marker và panel nút bấm
└─ urdf/ur3e_pen.urdf.xacro	UR3e gắn thêm cây bút vào tool0
└─ worlds/pen_board.sdf	nền và tấm bảng trắng
└─ test/	tám test hình học và test URDF

2.2. Sơ đồ khối các thành phần

Ba nhóm màu trong sơ đồ tương ứng ba tầng: màu xanh dương là mã do sinh viên viết, màu xanh lá là thành phần có sẵn của ROS 2 và MoveIt, màu cam là phần mô phỏng và hiển thị.



Hình 1. Sơ đồ khối các thành phần và hướng dữ liệu chạy giữa chúng.

2.3. Luồng hoạt động khi chạy

Khi gõ lệnh `ros2 launch ur_pen_plotter demo_all.launch.py`, chương trình đi qua các bước sau:

- Launch mở Ignition Gazebo với thế giới gồm nền, tấm bảng vẽ và robot UR3e có gắn bút; đồng thời nạp bộ điều khiển ros2_control, khởi động MoveIt và RViz.
- Sau 25 giây, node draw_shapes_node được chạy. Node đọc tham số từ canvas.yaml rồi dựng danh sách waypoint hai chiều cho cả ba hình.
- Node kiểm tra bố trí — khoảng hở giữa các hình và tấm với của cổ tay. Sai thì dừng ngay, chưa gửi lệnh nào xuống robot.
- Node chờ bộ điều khiển quỹ đạo sẵn sàng, sau đó đưa robot ra khỏi cấu hình kỳ dị ban đầu.
- Với từng nét, node đổi waypoint sang pose của tool0, gửi cho MoveIt lập kế hoạch rồi thi hành; trong lúc vẽ, một timer đọc vị trí thật của đầu bút để dựng nét mực và ghi CSV.
- Vẽ xong cả ba hình, robot xoay khớp vai 90 độ đổ sang bên để không che bản vẽ.

3. Các node và package chính

Toàn bộ mã do sinh viên viết nằm trong một package duy nhất là ur_pen_plotter. Các package mô phỏng của Universal Robots dùng bản cài sẵn từ apt chứ không chép vào repo, nên khi cập nhật gói thì thông số robot cũng cập nhật theo.

Thành phần	Loại	Nhiệm vụ
draw_shapes_node	Node C++	Node điều khiển chính. Dựng waypoint, kiểm tra bố trí, gửi MoveIt lập kế hoạch, thi hành quỹ đạo và ghi lại vết bút
stroke_preview	Node C++	Kiểm tra bố trí và xuất quỹ đạo dự định ra CSV; chạy được khi chưa bật Gazebo nên thử nhanh mỗi lần đổi tham số
canvas.cpp / hpp	Thư viện	Đổi toạ độ (u, v) trên mặt phẳng vẽ thành pose của tool0 và ngược lại
strokes.cpp / hpp	Thư viện	Sinh tập waypoint cho đường tròn, chữ V và chữ A
layout.cpp / hpp	Thư viện	Đo khoảng hở giữa các hình và tấm với, xuất CSV quỹ đạo dự định
ink_trail.cpp / hpp	Thư viện	Lấy mẫu TF đầu bút mỗi 25 ms, dựng Marker cho RViz và ghi CSV
demo_all.launch.py	Launch	Chạy trọn bộ bằng một lệnh
bringup_sim.launch.py	Launch	Chỉ dựng môi trường mô phỏng
draw.launch.py	Launch	Chỉ chạy node điều khiển khi môi trường đã sẵn sàng
canvas.yaml	Cấu hình	Toàn bộ hình học bản vẽ và tham số thực thi
sim_controllers.yaml	Cấu hình	Khai báo controller và nối dung sai quỹ đạo cho Gazebo
ur3e_pen.urdf.xacro	Mô hình	UR3e gắn thêm cây bút dài 10 cm vào mặt bích tool0
pen_board.sdf	Thế giới	Nền và tấm bảng trắng dựng tại $x = 0,40$ m

4. Cách tạo quỹ đạo

4.1. Hệ toạ độ mặt phẳng vẽ và phép ánh xạ

Mặt phẳng vẽ dựng đứng, vuông góc với trục X của hệ world và cách đế robot một khoảng $plane_x$. Trên mặt phẳng dùng một hệ hai chiều riêng: trục u chạy theo chiều $-Y$ của world, trục v chạy theo $+Z$, gốc toạ độ đặt ở độ cao z_0 . Một điểm (u, v) được đổi thành pose của tool0 theo công thức:

$$x = plane_x - pen_length - lift, \quad y = -u, \quad z = z_0 + v$$

$$q = (w = \sqrt{0.5}, q_x = 0, q_y = \sqrt{0.5}, q_z = 0)$$

Trong đó lift là khoảng nhấc bút, bằng 0 khi đang vẽ và bằng 5 cm khi di chuyển giữa hai nét. Số hạng pen_length xuất hiện vì thứ phải chạm mặt phẳng là đầu bút chứ không phải mặt bích; cây bút là một link thật trong URDF nên hình ảnh trong Gazebo và phép tính hoàn toàn khớp nhau. Quaternion cố định tương ứng phép quay 90 độ quanh trục Y, đưa trục Z của tool0 chĩa theo $+X$ của world, tức bút vuông góc với mặt giấy.

4.2. Đường tròn

Đường tròn tâm (u_c, v_c) bán kính R được quét từ đỉnh trên theo chiều kim đồng hồ trọn một vòng. Số điểm N tính theo chu vi chia cho bước lấy mẫu $\Delta s = 2$ mm:

$$\alpha_i = \frac{\pi}{2} - \frac{2\pi i}{N}, \quad i = 0 \dots N, \quad N = \left\lceil \frac{2\pi R}{\Delta s} \right\rceil$$

$$u_i = u_c + R \cos \alpha_i, \quad v_i = v_c + R \sin \alpha_i$$

Vì góc chạy đúng 2π nên điểm cuối trùng điểm đầu, nét khép kín mà không cần xử lý thêm.

4.3. Chữ V và khung chữ A

Các đoạn thẳng đều được nội suy tuyến tính giữa hai đầu mút, số điểm cũng lấy theo chiều dài chia cho Δs nên mật độ điểm đồng đều trên toàn bản vẽ:

$$p(t) = P_0 + (P_1 - P_0)t, \quad t = \frac{i}{n}, \quad n = \left\lceil \frac{\|P_1 - P_0\|}{\Delta s} \right\rceil$$

Chữ V nối ba điểm trên-trái → đỉnh nhọn dưới → trên-phải trong một nét liền. Khung chữ A cũng là hai đoạn nhưng lật ngược: dưới-trái → đỉnh trên → dưới-phải.

4.4. Thanh ngang của chữ A

Thanh ngang phải cắt đúng hai cạnh xiên. Gọi W là bề rộng chân chữ, H là chiều cao và k là tỉ lệ độ cao của thanh ngang tính từ đáy (mặc định $k = 0,4$). Vì hai cạnh thu hẹp tuyến tính từ chân lên đỉnh nên nửa bề rộng còn lại tại độ cao đó là:

$$b(k) = \frac{W}{2} (1 - k), \quad v_{bar} = v_c - \frac{H}{2} + kH$$

Hai nét của chữ A cắt nhau, nên mỗi nét được gán thêm tên hình mà nó thuộc về; phép kiểm tra khoảng hở ở mục sau chỉ áp dụng giữa các hình khác nhau, không bắt lỗi hai nét

trong cùng một chữ.

4.5. Kiểm tra bố trí trước khi chạy

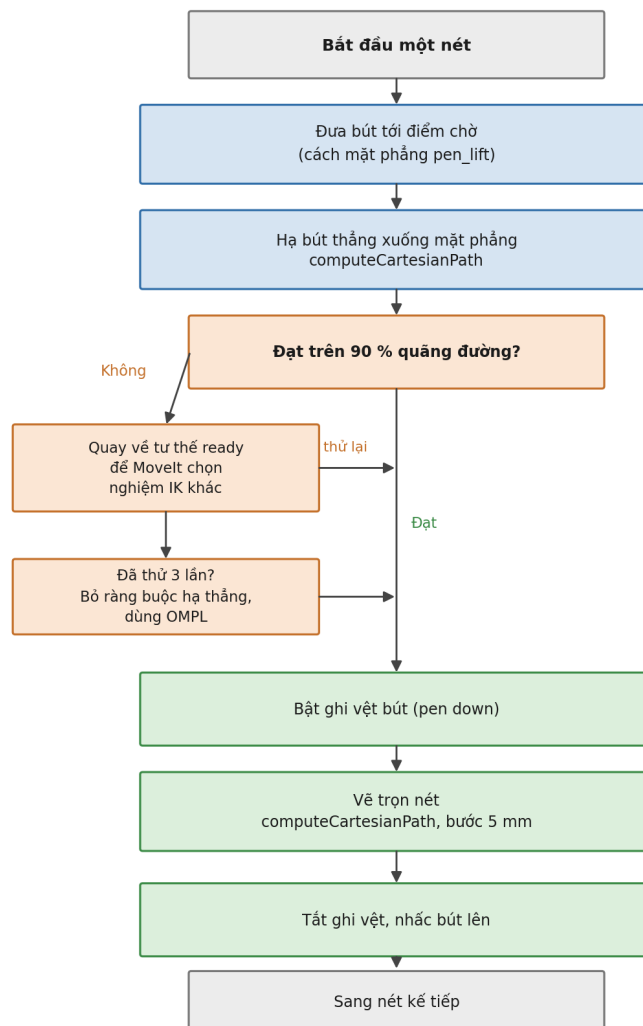
Trước khi gửi bất kỳ lệnh nào xuống robot, chương trình duyệt mọi cặp nét thuộc hai hình khác nhau để tính khoảng cách nhỏ nhất giữa hai tập điểm, đồng thời tính tầm với lớn nhất của tool0. Tầm với được đo từ trục khớp vai chứ không phải từ gốc tọa độ dưới sàn, vì thông số 0,50 m của UR3e là tính từ vai:

$$gap = \min_{i,j} \|p_i - q_j\| \geq gap_{min}$$

$$reach = \sqrt{x^2 + y^2 + (z - z_{vai})^2} \leq reach_{max}, \quad z_{vai} = 0.1519 \text{ m}$$

Ngưỡng mặc định là $gap_{min} = 30 \text{ mm}$ và $reach_{max} = 0,45 \text{ m}$. Sai một trong hai điều kiện thì chương trình in ra con số đo được rồi dừng hẳn — nhờ vậy lỗi bố trí bị bắt trong một giây thay vì phát hiện sau khi robot đã vẽ hỏng.

4.6. Trình tự vẽ một nét



Hình 2. Lưu đồ xử lý một nét vẽ, kể cả nhánh thử lại khi đường hạ bút bị chặn.

5. MoveIt lập kế hoạch và gửi trajectory tới robot

Node điều khiển làm việc với MoveIt 2 qua lớp MoveGroupInterface trên nhóm khớp `ur_manipulator`. Hai kiểu lập kế hoạch được dùng cho hai mục đích khác nhau:

Tình huống	Cách lập kế hoạch	Lý do chọn
Thoát điểm kỳ dị, tiếp cận điểm chờ của nét đầu tiên, đỡ tay sau khi vẽ xong	<code>setJointValueTarget</code> hoặc <code>setPoseTarget</code> , rồi <code>plan()</code> và <code>execute()</code> — bộ lập kế hoạch OMPL	Đây là chuyển động tự do trong không gian, hướng của tool thay đổi nhiều và không cần đi theo đường thẳng
Hạ bút, vẽ nét, nhắc bút, trượt ngang sang nét kế tiếp	<code>computeCartesianPath</code> với bước nội suy 5 mm, bật kiểm tra va chạm, rồi <code>execute()</code>	Nét vẽ bắt buộc bám đúng hình nên phải nội suy thẳng trong không gian Cartesian; hàm trả về tỉ lệ quãng đường đi được, dưới 90 % thì coi như thất bại

Kết quả của cả hai đường đều là một `moveit_msgs/RobotTrajectory`, tức chuỗi vị trí khớp kèm mốc thời gian. MoveIt gửi quỹ đạo này xuống `joint_trajectory_controller` qua action `/joint_trajectory_controller/follow_joint_trajectory`. Bộ điều khiển chạy bên trong Gazebo nhờ plugin `ign_ros2_control` khai báo trong URDF và điều khiển robot bằng giao diện vị trí. Node chỉ bắt đầu vẽ sau khi action server đã sẵn sàng, tránh gửi lệnh lúc mô phỏng chưa nạp xong.

6. Những vấn đề gặp phải và cách giải quyết

Phần lớn công sức làm bài không nằm ở việc sinh hình mà ở các vấn đề của mô phỏng và của bộ lập kế hoạch. Bảng dưới tóm tắt các lỗi thực tế đã gặp khi chạy và cách xử lý.

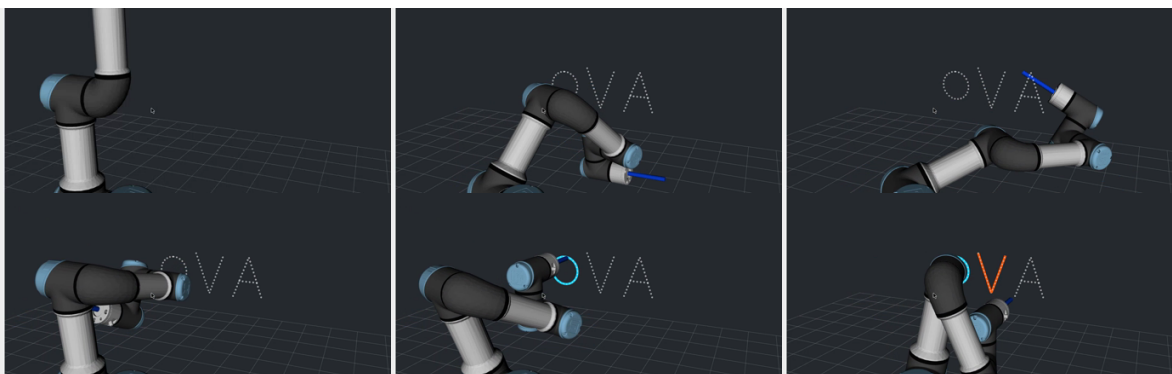
Hiện tượng	Nguyên nhân	Cách giải quyết
Mọi lời gọi IK hỏng ngay khi mô phỏng vừa khởi động	Robot ở cấu hình toàn bộ khớp bằng 0, cánh tay duỗi thẳng, ma trận Jacobian suy biến	Việc đầu tiên node làm là đưa robot về tư thế gập $q = [0, -\pi/2, \pi/2, -\pi/2, -\pi/2, 0]$
Bộ điều khiển huỷ lệnh giữa nét vẽ	Trong Gazebo có trọng lực, giao diện vị trí bám trễ vài độ nên vượt dung sai mặc định 0,2 rad	Dùng file controller riêng của package với dung sai 10 rad, truyền vào launch bằng tham số <code>runtime_config_package</code> thay vì sửa file hệ thống
Đường hạ bút chỉ đi được một phần rồi dừng	MoveIt chọn nghiệm động học ngược ngẫu nhiên; có nghiệm khiến đường thẳng đứng cắt qua trạng thái tự va chạm	Quay về tư thế ready để MoveIt bốc nghiệm khác rồi tiếp cận lại, tối đa ba lần; vẫn không được thì bỏ ràng buộc hạ thẳng và để OMPL tự tìm đường tới điểm chạm
Kế hoạch bị từ chối với lỗi đường đi không hợp lệ	OMPL lấy mẫu ngẫu nhiên nên thỉnh thoảng trả về đường quét căng tay vào đế robot	Thử lập kế hoạch lại tối đa năm lần trước khi báo thất bại
Nét mực ghi lại méo hoàn toàn dù robot vẽ đúng	TransformListener dùng chung executor với MoveIt; khi RViz và phần ghi màn hình chạy cùng thì callback bị nghẽn, TF tụt hậu hơn một giây	Cho TransformListener luồng riêng, timer lấy mẫu vào nhóm callback riêng và node chạy <code>MultiThreadedExecutor</code> . Độ lệch đo được giảm từ 98 mm xuống 0,07 mm

Hiện tượng	Nguyên nhân	Cách giải quyết
Toàn bộ quá trình khởi động chết mà thông báo không nhắc gì tới URDF	Launch của ur_simulation_gz truyền chuỗi URDF vào tham số robot_description không bọc ParameterValue nên launch đọc chuỗi đó như YAML; một dấu hai chấm cuối dòng trong comment là đủ để hỏng	Tránh dấu hai chấm trong file xacro và thêm một test tự động bung xacro rồi thử đọc bằng YAML để chặn lỗi tái diễn

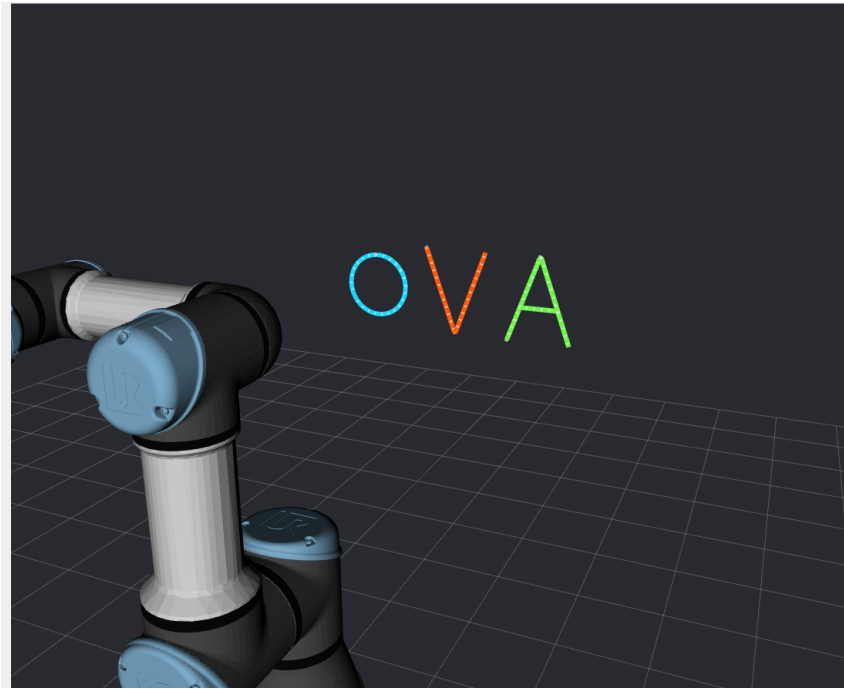
7. Kết quả chạy thử

Vị trí thực tế của đầu bút được lấy từ TF sau mỗi 25 ms, dựng thành nét mực trong RViz và ghi song song ra CSV. Nhờ đó sai lệch giữa quỹ đạo dự định và quỹ đạo robot đi thật được đo bằng số: với mỗi điểm thực tế, chương trình tìm điểm gần nhất trên quỹ đạo dự định của cùng một nét rồi thống kê.

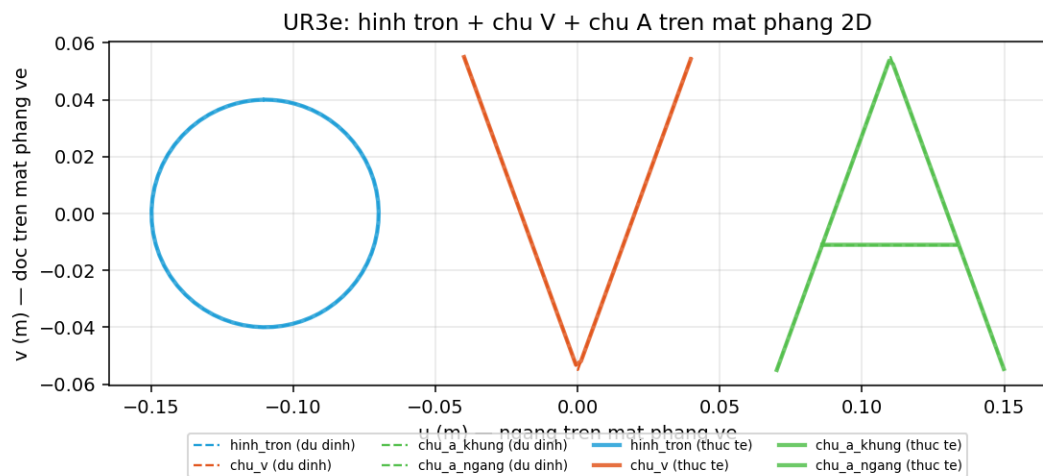
Chỉ số	Giá trị đo được
Khoảng hở nhỏ nhất giữa các hình	44,6 mm (tròn ↔ V); các cặp còn lại 66–156 mm
Tầm với lớn nhất của tool0	0,359 m so với giới hạn đặt 0,45 m
Chiều dài nét vẽ	tròn 25,1 cm · V 23,4 cm · khung A 23,4 cm · ngang A 4,8 cm
Tỉ lệ đường Cartesian tính được	100 % ở tất cả các đoạn
Sai số bám quỹ đạo, trung bình	0,46 – 0,52 mm tùy nét
Sai số bám quỹ đạo, lớn nhất	0,99 mm
Độ lệch đầu bút khỏi mặt phẳng vẽ	lớn nhất 0,07 mm
Kiểm thử đơn vị	8/8 test hình học đạt



Hình 3. Sáu thời điểm trong một lần chạy. Chuỗi điểm mờ là waypoint dự định, robot vẽ đè nét mực lên trên.



Hình 4. Kết quả cuối trong RViz. Robot đã đổ sang bên để không che bản vẽ.



Hình 5. Đối chiếu quỹ đạo dự định (nét đứt) với vết bút thực tế (nét liền) trong hệ toạ độ mặt phẳng vẽ.
Hai đường gần như trùng khít.

8. Hướng dẫn chạy nhanh

```
colcon build --symlink-install
source install/setup.bash
ros2 launch ur_pen_plotter demo_all.launch.py wait_for_button:=true
```

Với tham số `wait_for_button:=true`, robot đứng yên và các waypoint hiện sẵn thành chuỗi điểm trắng trên mặt phẳng vẽ cho tới khi bấm nút Next trong panel RvizVisualToolsGui của RViz — tiện khi cần quay video. Bỏ tham số này thì robot tự vẽ sau 25 giây. Muốn chạy tự động không cần màn hình thì dùng `./scripts/sim_test.sh`: script tự kiểm tra bố trí, bật mô phỏng trên màn hình ảo, vẽ, rồi in bảng sai số. Hướng dẫn cài đặt đầy đủ nằm trong README của repo.

Mục nội	Đường dẫn
Mã nguồn GitHub (public)	https://github.com/vanhhhhh512/ur3e_va
Video demo	(điền link Google Drive)